
WHY YOUR MODEL IS FAILING IN PRODUCTION

- By

Adnan Amin, Ph.D.

Assistant Professor / Program Director (CS)

School of Comp. Sci. & IT (IMSciences Peshawar)

Adnan.Amin@imsciences.edu.pk



WHY YOUR MODEL IS FAILING IN PRODUCTION

5 COUNTER-INTUITIVE TRUTHS ABOUT TRAINING DATA

- In machine learning, what works perfectly in the lab often fails in the real world.
- In school, training data is a cute little puppy. In production, It's a Kraken. Untamed, it will sink the entire ML operation



THE BIG PROBLEM:

CLASSROOM VS PRODUCTION

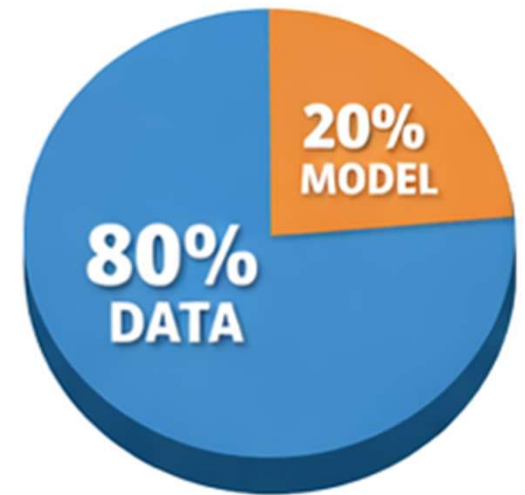
- In the lab, data behaves nicely. But in production, it becomes chaotic and dangerous.
- Lab = Clean, structured, small data
- Production = Messy, large, unpredictable data
- Models fail not because of algorithms... but data



KEY INSIGHT

DATA > MODEL

- ML success depends on data quality
- Not just model complexity
- Shift: Model-centric → Data-centric AI
- Most people focus on models, but in reality, data drives performance.



Data > Model

INTUITIVE TRUTHS ABOUT TRAINING DATA

5 COUNTER

1. Your features matter more than your algorithms
 2. Hand-labelling is bottleneck, not a gold standard
 3. The myth of static categories
 4. Data leakage is a subtle “Cheat Code”
 5. The “Power Law” of feature importance
-

FEATURES MATTER MORE THAN ALGORITHMS

- A simple model with strong features can outperform a deep learning model with poor data.
 - Feature engineering matters more than fancy algorithms.
 - Good features → simple models work
 - Bad features → complex models fail
 - Feature engineering = real power
-

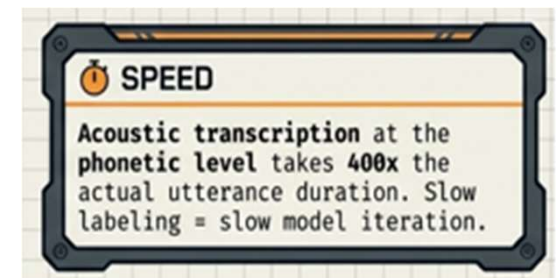
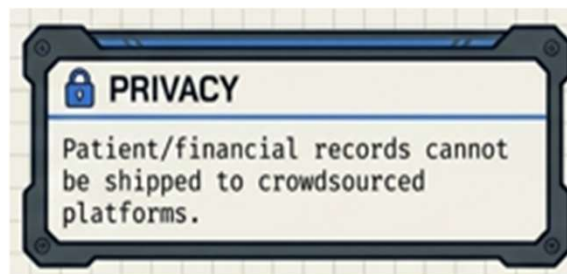
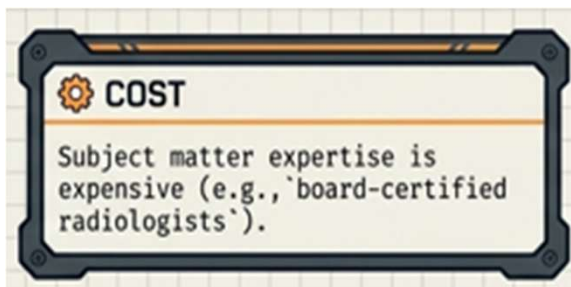
HAND-LABELING IS A BOTTLENECK

- Labeling data is often the slowest and most expensive part of ML.



- Attaching ground-truth meaning to raw inputs, either manually or programmatically.
 - Sol: instead of labeling manually, we can automate labeling using rules.
 - Use rules instead of manual labeling
 - Labeling functions
 - Scalable & reusable
-

THE HUMAN LABELING BOTTLENECK



- Ground truth is an illusion. Higher domain expertise often yields higher annotator disagreement.
 - Strict problem definitions and data lineage tracking are non-negotiable.
-

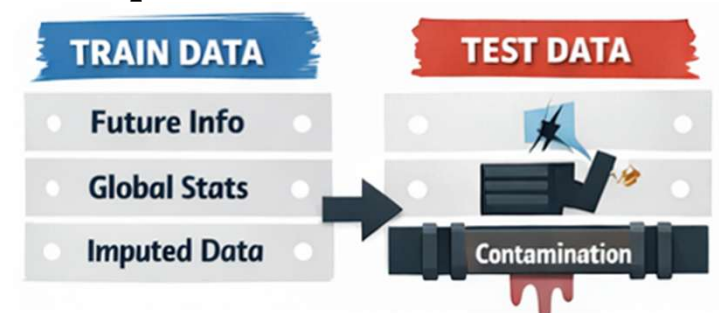
THE MYTH OF STATIC CATEGORIES

CATEGORIES ARE NOT STATIC

- In real stems, categories constantly change.
 - New users, brands, products appear
 - Model cannot handle unseen categories
- Handles unseen data, Slight accuracy loss, high stability.

DATA LEAKAGE = HIDDEN CHEAT CODE

- Leakage makes your model look perfect... until deployment.
 - Model sees future information
 - Unrealistically high accuracy
 - Fails in real-world
- Sol: Train vs test contamination, Perform preprocessing AFTER split.
- The key is to prevent your model from seeing the future.



POWER LAW OF FEATURES

NOT ALL FEATURES MATTER EQUALLY.

- Top 10 features → 50% impact
- Many features → minimal impact
- Too Many Features = Risk
 - More latency
 - More memory
 - More bugs
 - More leakage risk
- Sol: Focus on important features, Keep model simple, Reduce technical debt.
- A smaller, cleaner model is often more robust.



DATA-CENTRIC AI

DATA PIPELINE

- Focus on data quality
 - Engineer better features
 - Build robust pipelines
-

FINAL THOUGHT

- If ML is 80% data, why spend 90% time on code?
 - This mindset shift is the key to real-world ML success.
-

FEATURE ENGINEERING

ENGINEERED FEATURES

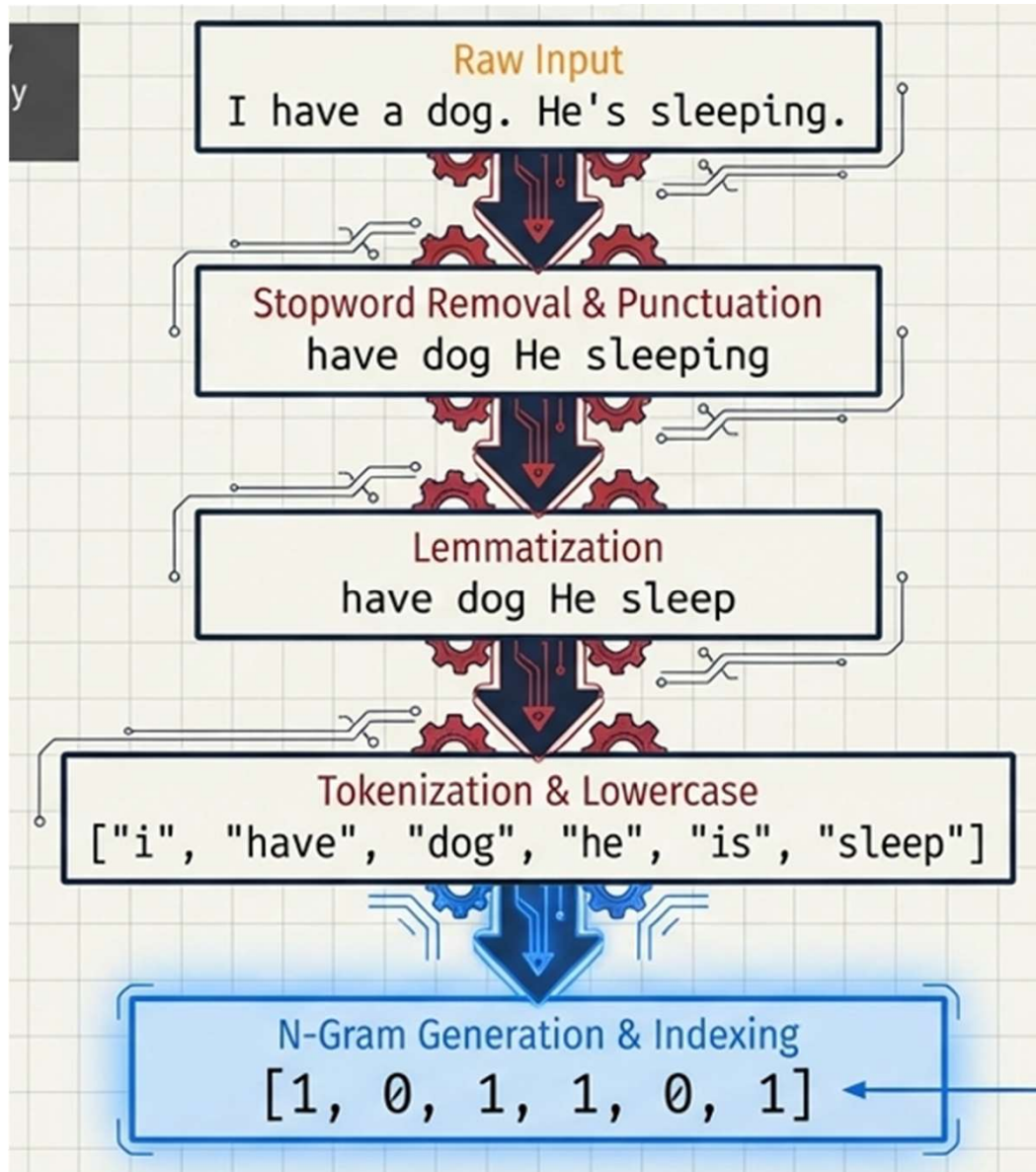
- State-of-the-art model architectures can still perform poorly if they don't use a good set of features.
 - In 2014, the paper [Practical Lessons from Predicting Clicks on Ads at Facebook](#) claimed that having the right features is the most important thing in developing their ML model.
 - Due to its importance, a large part of many ML engineering and data science jobs is to come up with new useful features.
-

QUESTIONS?

- Why do we have to worry about feature engineering?
 - Doesn't deep learning promise us that we no longer have to engineer features?
 - Deep learning = Features learning?
-

DOMAIN-SPECIFIC KNOWLEDGE

- Feature engineering requires knowledge of domain-specific techniques.
 - For example: the domain is natural language processing (NLP) and the target language is English.
-



FEATURES SELECTION VS ENGINEERING

- The process of choosing what features to use is called feature selection, and extracting the information you want to use is feature engineering.

COMMON FEATURE ENGINEERING OPERATIONS

- Handling missing values,
- Scaling, discretization,
- Encoding categorical features

HANDLING MISSING VALUES

NOT ALL TYPES OF MISSING VALUES ARE EQUAL

- **Missing Not at Random:** the value is missing because of the value itself (e.g., high-income earners refusing to disclose income)
 - **Missing at Random:** Missing due to another observed variable (e.g., gender A respondents hide their age).
 - **Missing Completely at Random:** No pattern, pure system glitch. Very rare.
 - **Solution:**
 - **Delete Row or Column:** Safe only for MCAR, if applied to MNAR/MAR, it actively removes vital predictive signals and severely biases the model.
 - **Imputation (Mean / Median/ Mode):** Fills gaps to save rows, but risks injecting artificial noise or creating catastrophic data leakage if calculated over the wrong data splits.
-

ISSUES WITH IMPUTATION

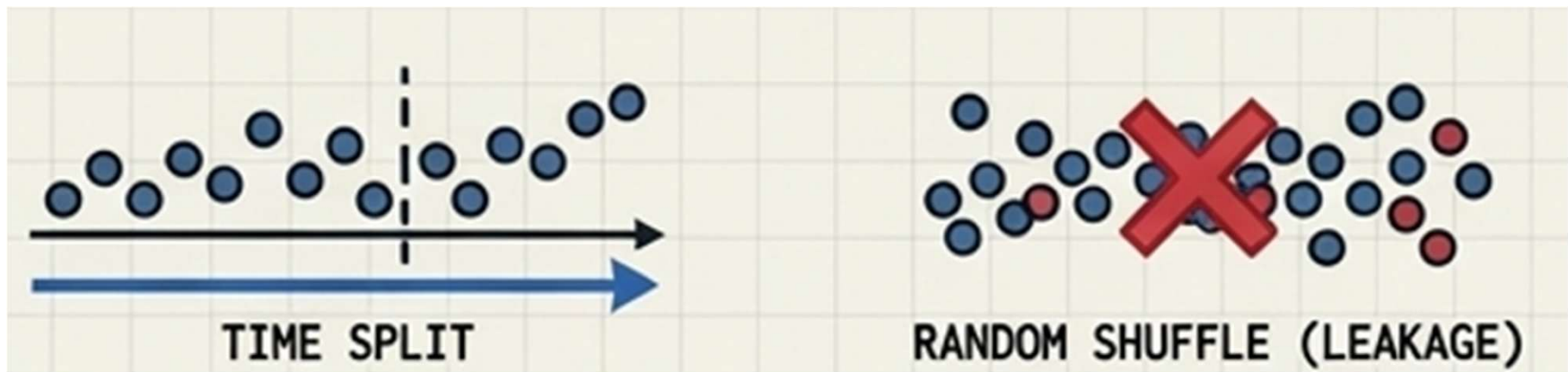
- With imputation, you risk adding noise to your data, or worse, data leakage.
 - **Data leakage:** when models perform beautifully on the test set but fail mysteriously in production.
-

DATA LEAKAGE

- Data leakage refers to the phenomenon when a form of the label "leaks" into the set of features used for making predictions, and this same information is not available during inference.
 - For example,
 - You obtained the data from hospital A, It did really well on the test data from hospital A, but poorly on the data from hospital B. later on it was confirmed that both hospital were using different scanning machine.
 - Solutions:
 - Split by time, not by random
 - Split before you scale.
 - Oversample after splitting
 - Ablation in your audit
-

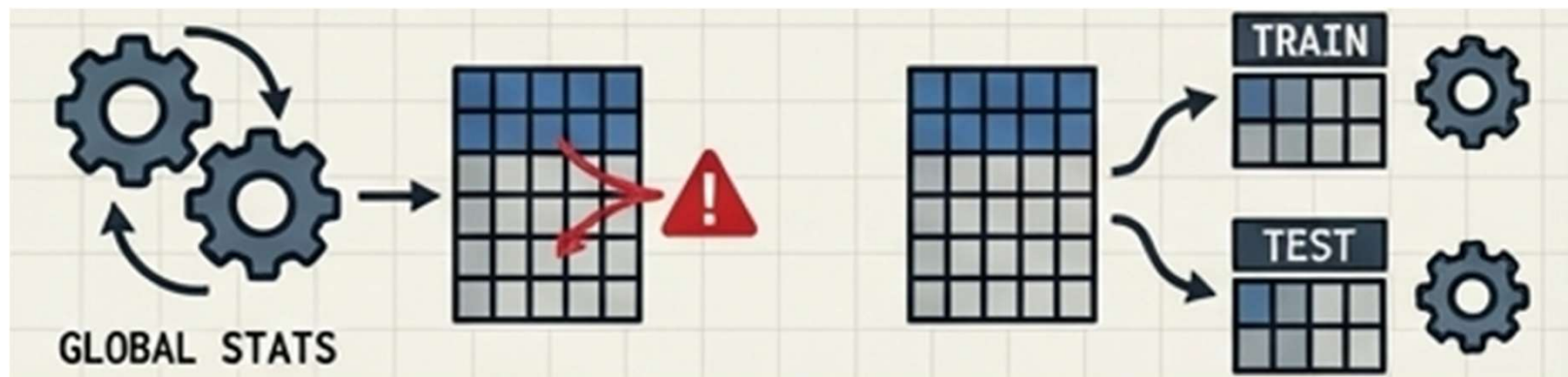
SPLIT BY TIME, NOT BY RANDOM

- If data is generated over time, random shuffling guarantees future information leakage



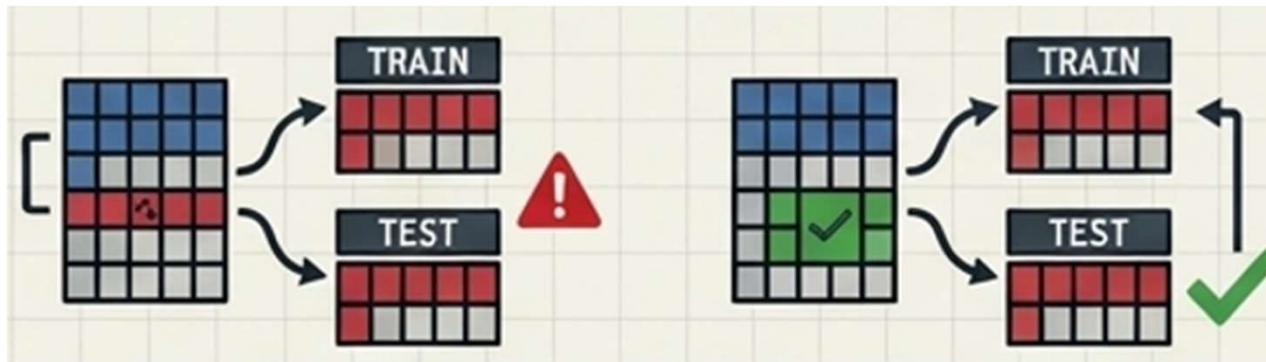
SPLIT BEFORE YOU SCALE.

- Calculating global mean/median across the entire dataset before splitting leaks test-set statistics into your training data.



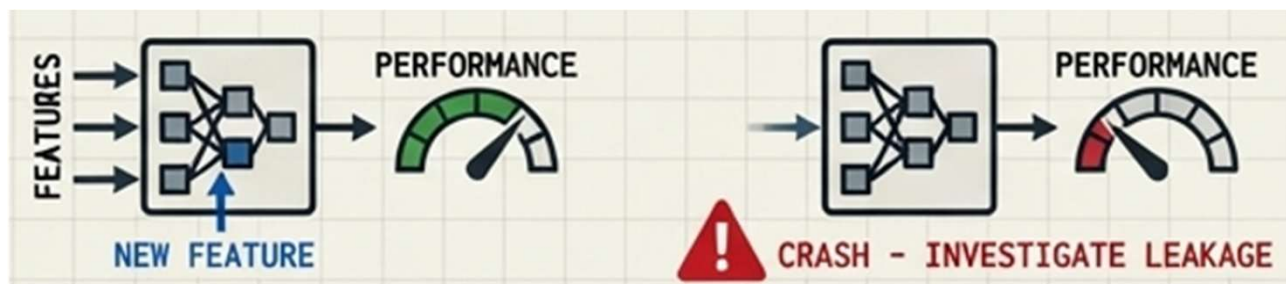
OVERSAMPLE AFTER SPLITTING

- Duplicating minority classes before splitting guarantees identical rows in both train and test sets.



ABLATION IS YOUR

- If removing a single new feature causes performance to crash, investigate it.
- It almost certainly contains leaked information.



FEATURE SCALING

- During data processing, it's important to scale your features so that they're in similar ranges. This process is called feature scaling.
 - This is one of the simplest things you can do that often result in a performance boost for your model
-

SCALING B/W (0,1): FORMULA

- An intuitive way to scale your features is to get each feature to be in the range [0, 1]. Given a variable x , its values can be rescaled to be in this range using the following formula.

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

- You can validate that if x is the maximum value, the scaled value x' will be 1. If x is the minimum value, the scaled value x' will be 0.
-

SCALING B/W ANY RANGE: FORMULA

- If you want your feature to be in an arbitrary range [a, b] -empirically, I find the range [-1, 1] to work better than the range [0, 1] -you can use the following formula.

$$x_{\hat{a}}^{TM} = a + \frac{(x - \min(x))(b - a)}{\max(x) - \min(x)}$$

DISCRETIZATION

- Discretization is the process of turning a continuous feature into a discrete feature. This process is also known as quantization. This is done by creating buckets for the given values.
 - For example, for annual income, you might want to group .
 - Lower income: less than \$35,000/year
 - Middle income: between \$35,000 and \$100,000/year
 - Upper income: more than \$100,000/year
 - The age variable is discrete,
 - Less than 18
 - Between 18 and 22
 - Between 22 and 30
 - Between 30 - 40
 - Between 40 - 65
 - Over 65
-

FEATURES CROSSING

- A technique to combine two or more features to generate new features.

ENCODING CATEGORICAL FEATURES

TURN CONTINUOUS FEATURES INTO CATEGORICAL FEATURES
